An Infix Operator and a Keyword Escape for C++

Steve Downey <sdowney@gmail.com>

Document #:	D4307R1
Date:	2026-10-11
Project:	Programming Language C++
Audience:	EWG
Source:	https://github.com/steve-downey/backtick papers/backtick-infix-and-keyword-escape.org `heads/blank-slate-0-g4e91cc5-dirty`

1 Abstract

2 Revision history

2.1 R1

Blank slate rewrite of prose.

2.2 R0

3 Before / After

3.1 The infix operator
3.1.1 Before
3.1.2 After
3.2 The keyword escape
3.2.1 Before
3.2.2 After

4 The proposal

5 Motivation, beyond readability

5.1 Short-circuiting returns to user code
5.2 One helper makes it a pipeline, and the ranges closures already fit
5.3 Stages with extra arguments: the honest gap
5.4 Reusable, point-free composition
5.5 Motivation, not a library proposal

6 Neither replaces the other: P2011's |>

7 Prior art

7.1 Infix application of named callables
7.2 Escaping keywords as identifiers

8 Design choices and decisions

8.1 It desugars to the call, and that is the whole design
8.2 Precedence: the highest binary operator, and why not higher
8.3 Left-associative
8.4 The operator slot is an assignment-expression
8.5 Bare nesting is chaining; real nesting takes parentheses
8.6 Evaluation order is the call's
8.7 A type name in the slot constructs
8.8 The escape yields an ordinary identifier
8.9 One spelling
8.10 Two uses, one paper; no library

9 Anticipated objections

9.1 "Define * on a type instead"

10 Fitting the grammar, without collision

10.1 The token is free
10.2 The productions
10.3 The same-delimiter problem
10.4 Operator and escape never meet

13.1 [lex.operators]

Modify the *operator-or-punctuator* grammar in [lex.operators] paragraph 1 by adding `:

operator-or-punctuator: one of
…
`?` `::` `.` `.*` `->` `->*` `^^` `~` `` ` ``
…

[*Note 1*: The row shown is [lex.operators]'s third; only the trailing ` is added. — *end note*]

13.2 [lex.name]

Add to [lex.name], after the paragraphs defining *identifier*:

escaped-identifier:
` *identifier* `

[x] An *escaped-identifier* may appear wherever the grammar uses *identifier*. The token between the ` tokens is treated as an *identifier* even if it would otherwise be a keyword or alternative token. The ` tokens are not part of the identifier. An *escaped-identifier* and an unescaped *identifier* with the same spelling are the same identifier.

[x+1]

[*Example 1*:

```
void `new`();          // declares a function named new
`new`();               // and calls it
int `count` = 0;
++count;               // names the same object
```

— *end example*]

13.3 [expr.mptr.oper]

Modify the grammar of [expr.mptr.oper] paragraph 1:

pm-expression:
~~*cast-expression*~~ *user-infix-expression*
pm-expression `.*` ~~*cast-expression*~~ *user-infix-expression*
pm-expression `->*` ~~*cast-expression*~~ *user-infix-expression*

13.4 [expr.backtick] (new subclause)

Annex A ([gram]) is updated mechanically to match.

Insert a new subclause between [expr.cast] and [expr.mptr.oper]:

Backtick operator [expr.backtick]

user-infix-expression:
cast-expression
user-infix-expression ` *backtick-operator* ` *cast-expression*

backtick-operator:
assignment-expression
simple-type-specifier
typename-specifier

[1] An expression of the form `E1 `O` E2` is identical (by definition) to `O(E1, E2)` ([expr.call]).

[2] If the *backtick-operator* is a *simple-type-specifier* or *typename-specifier* denoting a type `T` or a placeholder for a deduced class type, an expression of the form `E1 `T` E2` is identical (by definition) to `T(E1, E2)` ([expr.type.conv]). A *backtick-operator* whose tokens can be interpreted both as an *assignment-expression* and as a *simple-type-specifier* or *typename-specifier* is interpreted as a *simple-type-specifier* or *typename-specifier*.

[3] When parsing a *backtick-operator*, the first non-nested `[1] is taken as the ending delimiter, rather than as the first delimiter of a nested *user-infix-expression*.

1) A ` that appears within a matching pair of parentheses, brackets, or braces is nested.

14 Appendix A: alternative spellings, and the lexical inventory

14.1 A.1 Why alternatives get raised

14.2 A.2 The filter a candidate must pass

14.3 A.3 The candidates

14.4 A.4 The front-runner is not an alias

14.5 A.5 The rebuttals, pre-loaded

14.6 A.6 The wider inventory: what ASCII actually remains

14.7 A.7 Why the demand for that inventory evaporates